

CPSC 375 High-Performance Computing

Lecture 24

# The Sieve of Eratosthenes

## Implementation

# Block Decomposition Macros

```
#define BLOCK_LOW(id,p,n) \  
    ((i) * (n) / (p))
```

```
#define BLOCK_HIGH(id,p,n) \  
    BLOCK_LOW((id)+1,p,n)-1
```

```
#define BLOCK_SIZE(id,p,n) \  
    (BLOCK_LOW((id)+1) - BLOCK_LOW(id))
```

```
#define BLOCK_OWNER(index,p,n) \  
    (((p) * (index) + 1) - 1) / (n)
```

# Implementation (I)

```
#define MIN(a,b)  ((a)<(b)?(a):(b))

int main (int argc, char *argv[])
{
    ...
    MPI_Init (&argc, &argv);
    MPI_Barrier(MPI_COMM_WORLD);
    elapsed_time = -MPI_Wtime(); // to be added later
    MPI_Comm_rank (MPI_COMM_WORLD, &id);
    MPI_Comm_size (MPI_COMM_WORLD, &p);
    if (argc != 2) {
        if (!id) printf ("Command line: %s <m>\n", argv[0]);
        MPI_Finalize(); exit (1);
    }
}
```

# Implementation (2)

```
n = atoi(argv[1]);
low_value = 2 + BLOCK_LOW(id,p,n-1);
high_value = 2 + BLOCK_HIGH(id,p,n-1);
size = BLOCK_SIZE(id,p,n-1);
proc0_size = (n-1)/p;

marked = (char *) malloc(size);
if (marked == NULL) {
    printf ("Cannot allocate enough memory\n");
    MPI_Finalize();
    exit (1);
}

// all unmarked
for (i = 0; i < size; i++) marked[i] = 0;

/** End of Step 1 **/
```

# Implementation (3)

```
if (!id) index = 0;
prime = 2;          /** Step 2 **/

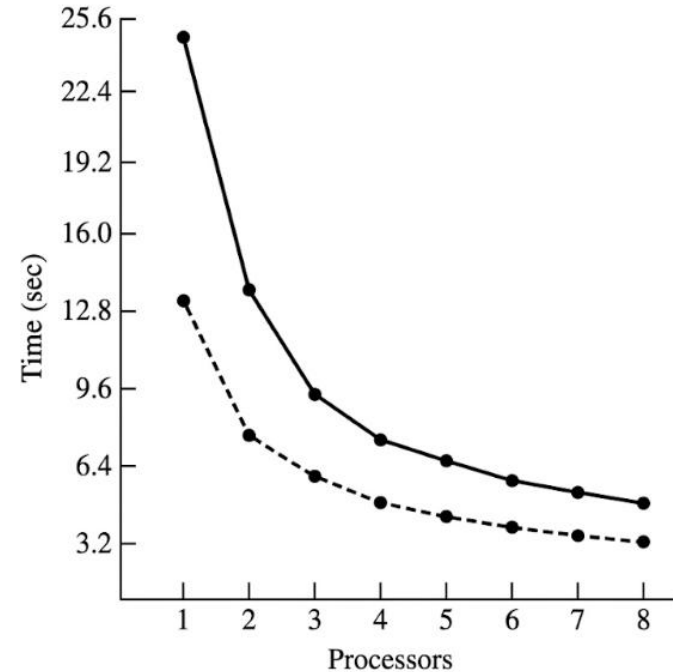
do {
    if (prime * prime > low_value) // determine where to begin
        first = prime * prime - low_value;           (Homework)
    else {
        if (!(low_value % prime)) first = 0;
        else first = prime - (low_value % prime);
    }
    for (i = first; i < size; i += prime) marked[i] = 1;
    if (!id) {
        while (marked[++index]);
        prime = index + 2;
    }
    MPI_Bcast (&prime, 1, MPI_INT, 0, MPI_COMM_WORLD);
} while (prime * prime <= n);
```

# Implementation (4)

```
count = 0;
for (i = 0; i < size; i++)
    if (!marked[i]) count++;
MPI_Reduce (&count, &global_count, 1, MPI_INT, MPI_SUM,
    0, MPI_COMM_WORLD);
elapsed_time += MPI_Wtime();
if (!id) {
    printf ("%d primes are <= %d\n", global_count, n);
    printf ("Total elapsed time: %10.6f\n", elapsed_time);
}
MPI_Finalize ();
return 0;
}
```

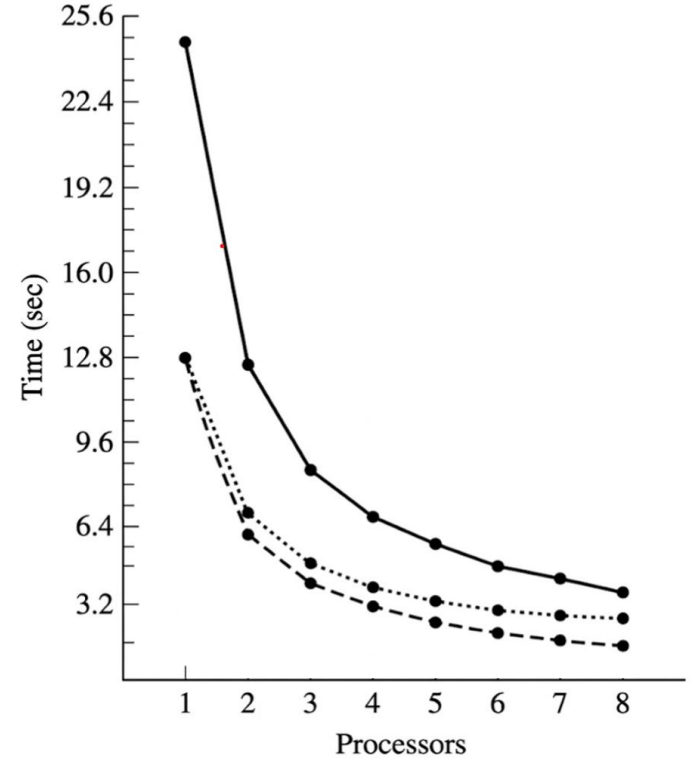
# Delete Even Integers

- Cuts the number of computations in half
- Frees storage for larger values of  $n$



# Eliminating Broadcast

- Each process finds its own sieving primes
  - Replicating the computation of primes to  $\sqrt{n}$
  - Eliminates the broadcast step





# Reorganizing Loops

- Each process is marking widely dispersed elements of a very large array, leading to a poor cache hit rate.
- Outer loop iterates over prime sieve values between 3 and  $\sqrt{n}$ .
- Inner loop iterates over the process's share of the integers between 3 and  $n$ .
- If we exchange the inner and outer loops, we can improve the cache hit rate.
  - Fill the cache with a section of the larger subarray, then mark it before bringing in the next section of the subarray.

# Reorganizing Loops

Finding primes between 3 and 99.

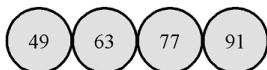
3-99: multiples of 3



3-99: multiples of 5



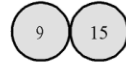
3-99: multiples of 7



- cache has two lines
- each line can hold 16 bytes
- one line contains integers 3, 5, 7, and 9
- the next line holds 11, 13, 15, and 17; etc.
- shaded circles represent cache misses.

# Reorganizing Loops

3-17: multiples of 3



19-33: multiples of 3, 5



35-49: multiples of 3, 5, 7



51-65: multiples of 3, 5, 7



67-81: multiples of 3, 5, 7



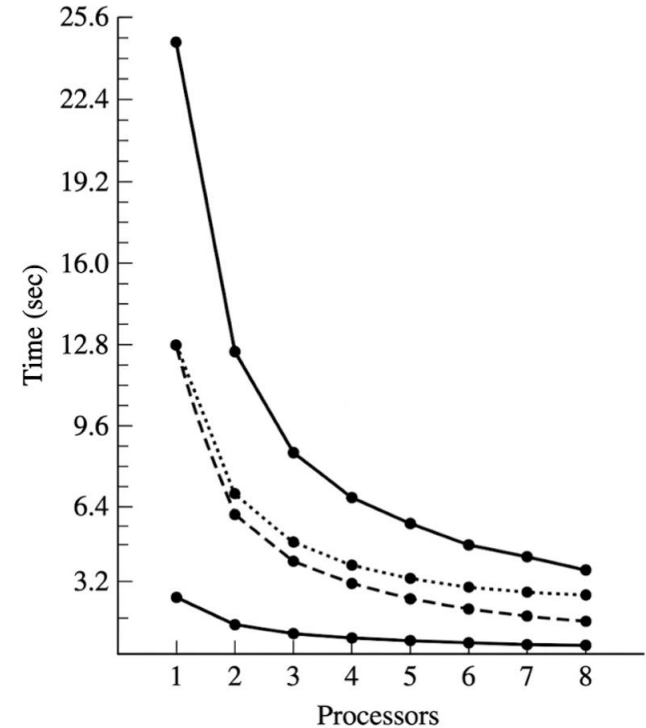
83-97: multiples of 3, 5, 7



99: multiples of 3, 5, 7

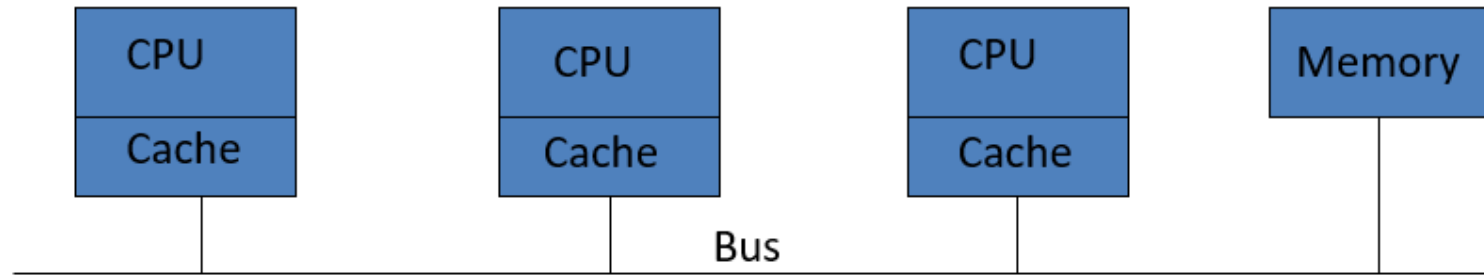


Fewer shaded circles indicate the cache hit rate has improved.





# Distributed Shared-Memory Systems



A multiprocessor with caching

# Write Through Protocol

| Event      | Action taken by a cache in response to its own CPU's operation | Action taken by a cache in response to a remote CPU's operation |
|------------|----------------------------------------------------------------|-----------------------------------------------------------------|
| Read miss  | Fetch data from memory and store in cache                      | no action                                                       |
| Read hit   | Fetch data from local cache                                    | no action                                                       |
| Write miss | Update data in memory and store in cache                       | no action                                                       |
| Write hit  | Update memory and cache                                        | invalidate cache entry                                          |

# Write Once Protocol

- This protocol manages cache blocks, each of which can be in one of the following three states:

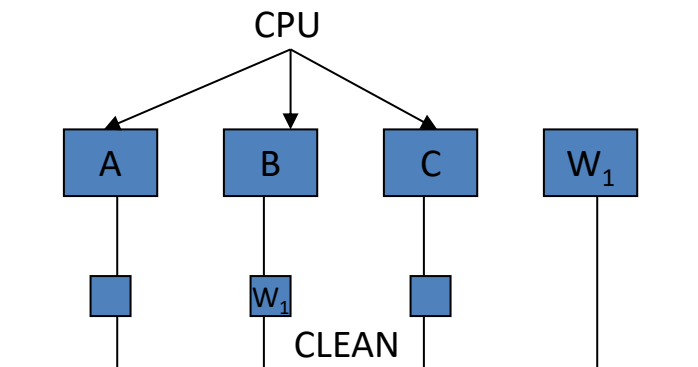
INVALID: This cache block does not contain valid data.

CLEAN: Memory is up-to-date; the block may be in other caches.

DIRTY: Memory is incorrect; no other cache holds the block.

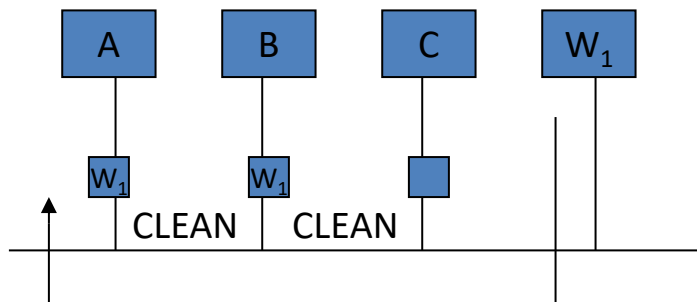
- The basic idea of the protocol is that a word that is being read by multiple CPUs is allowed to be present in all their caches.
- A word that is being heavily written by only one machine is kept in its cache and not written back to memory on every write to reduce bus traffic.

# Example



Memory is correct

Initial state – word  $W$  containing value  $W_1$  is in memory and is also cached by B.

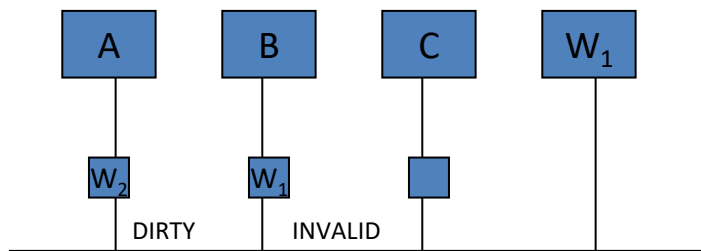


Memory is correct

A reads word  $W$  and gets  $W_1$ . B does not respond to the read, but the memory does.



# Example

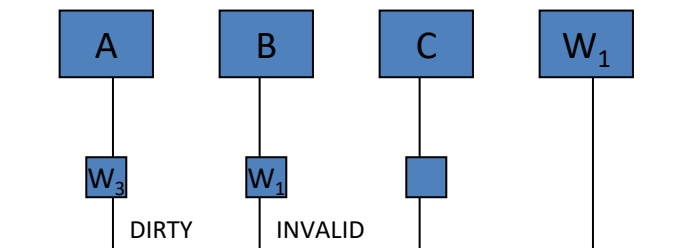


Memory is correct

A write a value W<sub>2</sub>, B snoops on the bus, sees the write, and invalidates its entry.

A's copy is marked DIRTY.

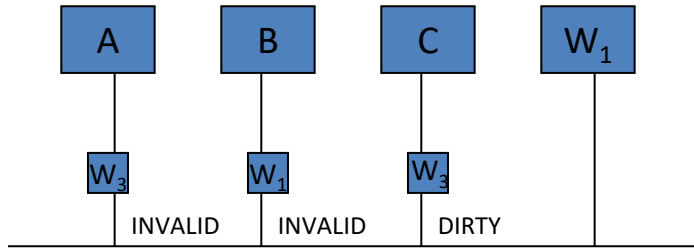
Not update memory



Memory is correct

A write W again. This and subsequent writes by A are done locally, without any bus traffic.

# Example



C reads or writes W. A sees the request by snooping on the bus, provides the value, and invalidates its own entry. C now has the only valid copy.

Not update memory

